

VOLUME 2 OF 2 · ENGINEERING

SADHANA

Technical implementation: architecture, schema, code, deployment, sprint plan

Foundations · Stack, architecture, database schema

Modules · Feed, exam hub, quiz, library, community

Operations · Scaling, security, CI/CD, 14-week sprints

మొదటి రోజు నుంచి లాంచ్ వరకు

పూర్తి సాంకేతిక ప్రణాళిక

Contents

How to use this document	5
Chapter 1 — Technology Decisions	5
1.1 The stack	5
1.2 Decisions explicitly rejected	6
1.3 Non-functional requirements	7
Chapter 2 — System Architecture	8
2.1 High-level architecture	8
2.2 Application structure — modular monolith	8
2.3 Environments	9
Chapter 3 — Repository Structure	10
Chapter 4 — Database Schema	12
4.1 Design principles	12
4.2 Core tables	12
4.3 Entity relationships	19
Chapter 5 — Multi-Language Implementation	20
5.1 Configuration	20
5.2 Routing	20
5.3 The locale middleware	21
5.4 Layer 1 — UI strings	21
5.5 Layer 2 — Editorial content	22
5.6 Layer 3 — User content, translated lazily	23
5.7 The glossary — do not skip this	23
5.8 Cache keys must include locale	24
5.9 Typography	24
Chapter 6 — Module: Notification Feed	26
6.1 The eligibility engine	26
6.2 Personalised feed query	28
6.3 Feed component	28
6.4 Notification card	29

Chapter 7 — Module: Exam Hub (the SEO engine)	31
7.1 Controller with aggressive caching	31
7.2 Page structure — fixed, in this order	31
7.3 SEO head	32
7.4 JobPosting structured data for notifications	33
7.5 Sitemaps	33

Chapter 8 — Module: Daily Quiz and Streaks	34
8.1 Automated daily publication	34
8.2 Streak service	35
8.3 Shareable result — the viral loop	36

Chapter 9 — Module: Material Library	37
9.1 Upload flow	37
9.2 Moderation queue rules	37
9.3 Mobile-readable versions	38

Chapter 10 — Module: Community	39
10.1 Search-before-ask	39
10.2 Reputation	39
10.3 Moderation ladder	40

Chapter 11 — Ingestion Pipeline	41
11.1 Architecture	41
11.2 Parser pattern	41
11.3 AI-assisted extraction	42
11.4 Source list (Phase 1)	43

Chapter 12 — Search	44
----------------------------	-----------

Chapter 13 — Notifications and Push	45
13.1 Channels	45
13.2 Frequency discipline	45
13.3 Batched dispatch	45

Chapter 14 — Performance and Scaling	47
14.1 Caching strategy	47

14.2 Scaling ladder	47
14.3 Notification-day playbook	48
14.4 Frontend performance	48
Chapter 15 — Security	49
15.1 Application security	49
15.2 DPDP Act compliance — build in v1	49
15.3 Content security	50
Chapter 16 — Testing	51
16.1 Priorities	51
16.2 Sample tests	51
16.3 Load testing	52
Chapter 17 — CI/CD and Deployment	53
17.1 CI	53
17.2 Deploy	53
17.3 Supervisor for queue workers	54
17.4 Scheduled tasks	54
17.5 Backups	54
Chapter 18 — Monitoring	55
Chapter 19 — Launch Checklist	56
Chapter 20 — Sprint Plan	58
Sprint 0 — Foundation (Weeks 1–2)	58
Sprint 1 — Exam Hub (Weeks 3–4)	58
Sprint 2 — Notifications (Weeks 5–7)	59
Sprint 3 — Quiz and Streaks (Weeks 8–10)	59
Sprint 4 — Polish and Launch (Weeks 11–14)	60
Post-launch cadence	60
Appendix A — Environment variables	61
Appendix B — Composer dependencies	62
Appendix C — Definition of Done	63

Field	Detail
Document	Volume 2 of 2 — Technical Implementation
Companion	Volume 1 — R&D and Business Case
Version	1.0
Date	September 2026
Audience	Engineering
Prerequisite	Read Volume 1 first

How to use this document

Volume 1 said *what* and *why*. This says *how*, in build order.

Chapters 1–5 are foundations you set up once. Chapters 6–13 are the modules, in the order they should be built. Chapters 14–19 are operational concerns you cannot bolt on later.

Chapter 20 is the sprint-by-sprint plan. If you only read one chapter before starting, read that one — then come back for the detail.

Code samples are illustrative and production-shaped, not copy-paste-complete. Validation, authorization, and error handling are shown where they carry a decision, and elided where they are routine.

Chapter 1 — Technology Decisions

సారాంశం: Laravel + Livewire + MySQL. తెలిసిన స్టాక్‌తో వేగంగా బిల్డ్ చేయడమే ముఖ్యం.

1.1 The stack

Layer	Choice	Why
Backend	Laravel 12 (PHP 8.3)	Founder's deepest expertise. Shipping speed beats theoretical fit. Mature i18n, queues, caching.

Layer	Choice	Why
Frontend	Livewire 3 + Alpine.js	Server-rendered HTML is essential for SEO — our entire acquisition strategy depends on Google indexing content. An SPA would require SSR complexity for no gain.
Admin	Filament 3	Content team needs a CRUD panel on day one. Filament gives it in days, not weeks.
Database	MySQL 8	JSON columns for translations, full-text where needed, well-understood scaling path
Cache / Queue	Redis	Streaks, leaderboards (sorted sets are perfect for this), sessions, job queue
Search	Meilisearch	Fast, typo-tolerant, per-language indexes, trivial to self-host. Elasticsearch is overkill at our scale.
Object storage	Cloudflare R2	S3-compatible, zero egress fees. PDF downloads at scale would bankrupt us on S3.
CDN / WAF	Cloudflare	Free tier handles our spike profile; caching rules are the difference between surviving notification day and not
Client	PWA first , Flutter app in Phase 3	No install friction, works on cheap phones, one codebase, instant updates
Push	Firebase Cloud Messaging + WhatsApp Cloud API	Web push for browsers; WhatsApp because it is the highest open-rate channel in our market
Hosting	Hostinger VPS → Hetzner / DigitalOcean at scale	Existing pipeline; migrate when we outgrow it
CI/CD	GitHub Actions	Existing pipeline, reuse it
Monitoring	Sentry + Laravel Pulse + Uptime Kuma	Errors, performance, availability

1.2 Decisions explicitly rejected

Rejected	Why
Next.js / React SPA	SEO is the business. Server-rendered HTML is non-negotiable, and adding SSR to React costs complexity we do not need.
Separate translations table	Joins on every content read, for a workload that is 95% reads. JSON columns are faster and simpler.
Elasticsearch	Operational weight far beyond our scale needs
Native app first	Install friction kills top-of-funnel; our users have limited storage on limited phones
Microservices	A team of two does not need distributed systems. Modular monolith, always.
Kubernetes	Same reason. Revisit above 2M MAU, if ever.

1.3 Non-functional requirements

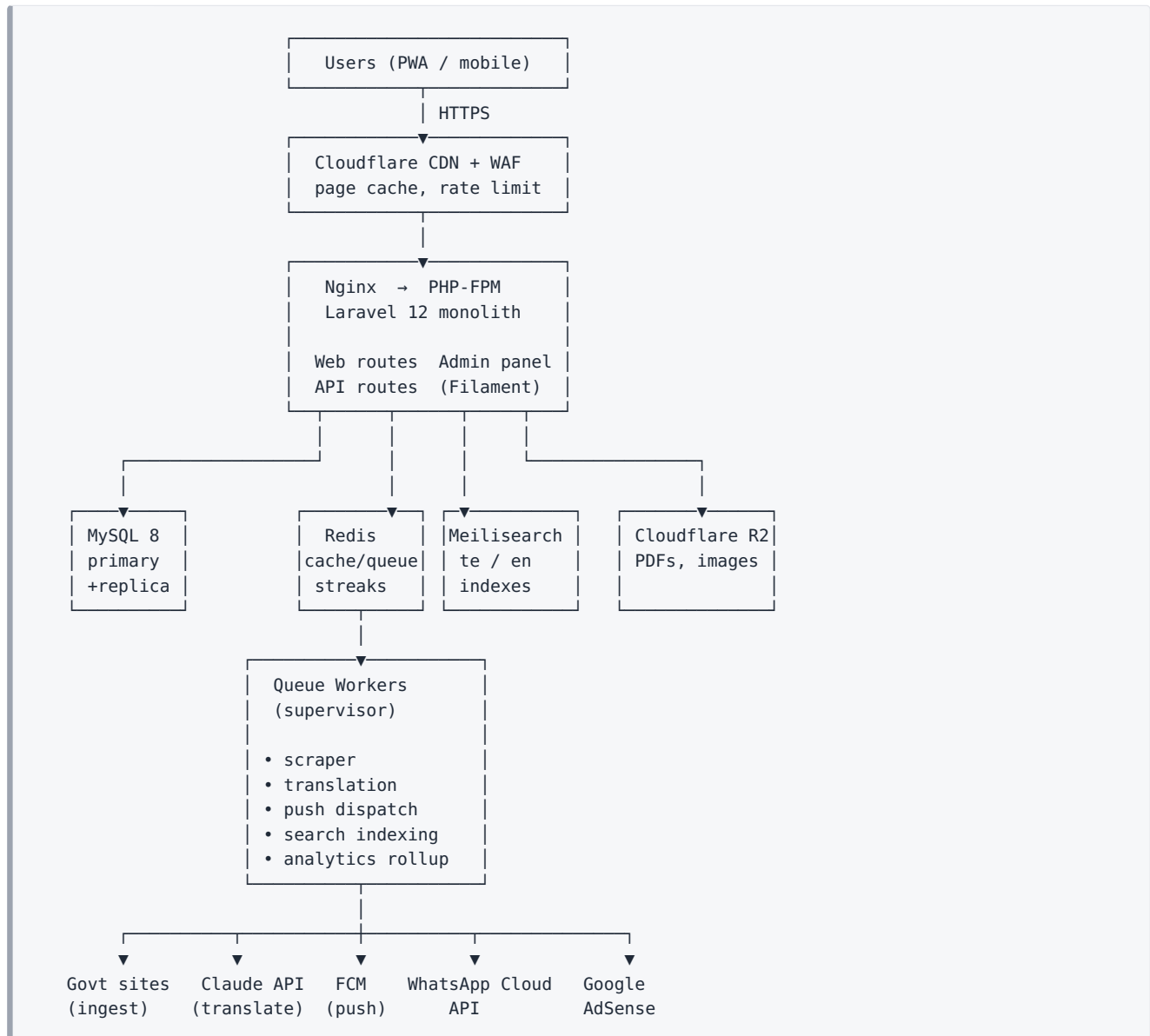
These are hard constraints. If a feature cannot meet them, redesign the feature.

Requirement	Target
Largest Contentful Paint on 3G, ₹8,000 Android	Under 2.5s
Time to Interactive	Under 3.5s
First-load JS payload	Under 100 KB gzipped
Server response (cached page)	Under 200ms p95
Server response (dynamic page)	Under 500ms p95
Uptime	99.5% normal, 99.9% on notification days
Spike capacity	50x baseline traffic without degradation
Works offline	Saved material and last-seen feed

Chapter 2 — System Architecture

సారాంశం: ఒకే యాప్, పక్కన బ్యాక్‌గ్రౌండ్ వర్క్‌లు. సింపుల్‌గా ఉంచడమే లక్ష్యం.

2.1 High-level architecture



2.2 Application structure — modular monolith

One Laravel application, organised into modules with clear boundaries. Modules talk through service classes and events, never by reaching into each other's models.

```

app/
├── Modules/
│   ├── Notification/    # job & exam notifications
│   ├── Exam/           # exam hubs, syllabus
│   ├── Quiz/           # daily quiz, streaks
│   ├── Material/       # library, uploads
│   ├── Community/      # doubts, answers
│   ├── TestSeries/     # mocks, analytics
│   ├── Localization/   # translation pipeline
│   └── Advertising/    # ad slots, tracking
├── Models/
├── Services/
├── Jobs/
├── Events/
└── Support/

```

The rule: if `Community` needs exam data, it calls `ExamService`, not `Exam::find()`. This keeps the option open to extract a module into its own service later, and — more immediately — keeps the codebase understandable when it is 40,000 lines.

2.3 Environments

Environment	Purpose	Data
local	Development (Laravel Sail / Herd)	Seeded fixtures
staging	Pre-release verification	Anonymised production copy
production	Live	Real

Every environment runs identical PHP, MySQL, and Redis versions. Version drift between staging and production is how "it worked on staging" incidents happen.

Chapter 3 — Repository Structure

సారాంశం: ఫోల్డర్ నిర్మాణం మొదట్లోనే సరిచేస్తే తరువాత గందరగోళం ఉండదు.

```
sadhana/
├── app/
│   ├── Console/Commands/
│   │   ├── ScrapeNotifications.php
│   │   ├── PublishDailyQuiz.php
│   │   └── RollupAnalytics.php
│   ├── Filament/                                # admin panel resources
│   │   ├── Resources/
│   │   └── Pages/
│   ├── Http/
│   │   ├── Controllers/
│   │   ├── Middleware/
│   │   │   ├── SetLocale.php
│   │   │   └── TrackAdImpression.php
│   │   └── Requests/
│   ├── Livewire/                                # UI components
│   │   ├── Feed/
│   │   ├── Quiz/
│   │   ├── Community/
│   │   └── Exam/
│   ├── Jobs/
│   │   ├── TranslateContent.php
│   │   ├── SendPushNotification.php
│   │   ├── IndexSearchable.php
│   │   └── ScrapeSource.php
│   ├── Models/
│   ├── Modules/
│   ├── Services/
│   │   ├── EligibilityService.php
│   │   ├── StreakService.php
│   │   ├── TranslationService.php
│   │   └── ScraperService.php
│   └── Support/
│       ├── Locale.php
│       └── SeoBuilder.php
├── config/
│   ├── locales.php                            # supported languages, config-driven
│   ├── exams.php
│   └── scrapers.php
├── database/
│   ├── migrations/
│   ├── seeders/
│   └── factories/
├── lang/
│   ├── en.json
│   ├── te.json
│   └── hi.json
├── public/
│   ├── fonts/                                # self-hosted, subset Noto Sans Telugu
│   ├── sw.js                                # service worker
│   └── manifest.json                         # PWA manifest
├── resources/
│   ├── views/
│   │   ├── layouts/
│   │   └── components/
```

```
| | | exam/
| | | feed/
| | | quiz/
| | |
| | | css/
| | | js/
| | |
| | | routes/
| | | web.php
| | | api.php
| | | console.php
| | |
| | | tests/
| | | Feature/
| | | Unit/
| | |
| | | .github/workflows/
| | | ci.yml
| | | deploy.yml
```

Chapter 4 — Database Schema

సారాంశం: ఇదే ప్రాజెక్ట్ పునాది. ఇక్కడ తప్పు చేస్తే తర్వాత అంతా కష్టం.

4.1 Design principles

1. **Translatable content lives in JSON columns**, not in a separate table. Shape: `{"en": "...", "te": "..."} .`
2. **Slugs are never translated**. One slug per entity across all languages. Translating slugs fragments backlinks and doubles the sitemap for no SEO gain.
3. **Soft deletes** on all user-facing content — accidental deletion of an exam page loses SEO position permanently.
4. **UUIDs on public-facing IDs** for user content; auto-increment internally. Do not leak record counts.
5. **Every table has `created_at` / `updated_at`** . No exceptions.

4.2 Core tables

Users and profile

```
CREATE TABLE users (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  uuid        CHAR(36) UNIQUE NOT NULL,
  name        VARCHAR(120) NOT NULL,
  phone       VARCHAR(15) UNIQUE NOT NULL,
  phone_verified_at  TIMESTAMP NULL,
  email       VARCHAR(190) UNIQUE NULL,
  password    VARCHAR(255) NULL,          -- nullable: OTP-first auth
  preferred_locale  CHAR(5) DEFAULT 'te',
  reputation  INT UNSIGNED DEFAULT 0,
  is_verified_selected  TINYINT(1) DEFAULT 0,    -- cleared an exam
  last_active_at  TIMESTAMP NULL,
  created_at      TIMESTAMP NULL,
  updated_at      TIMESTAMP NULL,
  deleted_at      TIMESTAMP NULL,
  INDEX idx_last_active (last_active_at),
  INDEX idx_reputation (reputation DESC)
);

CREATE TABLE profiles (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NOT NULL,
  date_of_birth  DATE NULL,                -- drives age eligibility
  gender      ENUM('male','female','other') NULL,
  category    ENUM('general','obc','sc','st','ews') NULL,  -- age relaxation
  is_pwd      TINYINT(1) DEFAULT 0,
  is_ex_serviceman  TINYINT(1) DEFAULT 0,
```

```

highest_qualification ENUM('10th','12th','iti','diploma','degree',
                           'pg','btech','mbbs','phd') NULL,
qualification_stream VARCHAR(100) NULL,
state                 CHAR(2) DEFAULT 'TS',
district              VARCHAR(60) NULL,
created_at            TIMESTAMP NULL,
updated_at            TIMESTAMP NULL,
UNIQUE KEY uk_user (user_id),
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
INDEX idx_eligibility (highest_qualification, category, date_of_birth)
);

CREATE TABLE user_exam_preferences (
  user_id      BIGINT UNSIGNED NOT NULL,
  exam_id      BIGINT UNSIGNED NOT NULL,
  priority     TINYINT DEFAULT 1,
  created_at   TIMESTAMP NULL,
  PRIMARY KEY (user_id, exam_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE CASCADE
);

```

Exams

```

CREATE TABLE exam_categories (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  slug        VARCHAR(80) UNIQUE NOT NULL,      -- 'state-psc', 'railway'
  name        JSON NOT NULL,                    -- {"en":"State PSC","te":"..."}
  icon        VARCHAR(60) NULL,
  sort_order  INT DEFAULT 0,
  created_at  TIMESTAMP NULL,
  updated_at  TIMESTAMP NULL
);

CREATE TABLE exams (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  category_id BIGINT UNSIGNED NOT NULL,
  slug        VARCHAR(120) UNIQUE NOT NULL,    -- 'tgpsc-group-2'
  name        JSON NOT NULL,
  short_name  VARCHAR(60) NOT NULL,
  conducting_body VARCHAR(120) NOT NULL,
  official_website VARCHAR(255) NULL,
  description  JSON NULL,
  eligibility_summary JSON NULL,
  exam_pattern JSON NULL,                      -- structured, per locale
  syllabus    JSON NULL,                      -- structured, per locale
  meta_title   JSON NULL,
  meta_description JSON NULL,
  faq          JSON NULL,                     -- schema.org FAQPage
  state        CHAR(2) NULL,                  -- NULL = national
  is_active    TINYINT(1) DEFAULT 1,
  view_count   BIGINT UNSIGNED DEFAULT 0,
  created_at   TIMESTAMP NULL,
  updated_at   TIMESTAMP NULL,
  deleted_at   TIMESTAMP NULL,
  FOREIGN KEY (category_id) REFERENCES exam_categories(id),
  INDEX idx_active_state (is_active, state),
  INDEX idx_views (view_count DESC)
);

CREATE TABLE exam_cutoffs (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  exam_id     BIGINT UNSIGNED NOT NULL,
  year        SMALLINT NOT NULL,

```

```

category    VARCHAR(20) NOT NULL,
cutoff_marks DECIMAL(6,2) NULL,
total_marks DECIMAL(6,2) NULL,
notes       JSON NULL,
source_url   VARCHAR(500) NULL,
created_at   TIMESTAMP NULL,
updated_at   TIMESTAMP NULL,
FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE CASCADE,
INDEX idx_exam_year (exam_id, year DESC)
);

```

Notifications — the heart of the product

```

CREATE TABLE notifications (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  uuid        CHAR(36) UNIQUE NOT NULL,
  exam_id     BIGINT UNSIGNED NULL,
  slug        VARCHAR(200) UNIQUE NOT NULL,
  title       JSON NOT NULL,
  description  JSON NULL,
  organisation VARCHAR(160) NOT NULL,
  job_type     ENUM('government','private','psu','contract') DEFAULT 'government',

  -- vacancy
  total_vacancies INT UNSIGNED NULL,
  vacancy_breakdown JSON NULL,          -- by post and category

  -- eligibility (drives the matching engine)
  min_qualification ENUM('10th','12th','iti','diploma','degree',
                        'pg','btech','mbbs','phd') NULL,
  qualification_notes JSON NULL,
  min_age           TINYINT UNSIGNED NULL,
  max_age           TINYINT UNSIGNED NULL,
  age_relaxation     JSON NULL,          -- {"obc":3,"sc":5,"st":5,"pwd":10}
  age_reference_date DATE NULL,
  allowed_states     JSON NULL,          -- ["TS"] or null for all-India
  allowed_districts  JSON NULL,
  gender_restriction ENUM('any','male','female') DEFAULT 'any',

  -- dates
  notification_date DATE NULL,
  apply_start_date  DATE NULL,
  apply_end_date    DATE NULL,
  exam_date         DATE NULL,
  admit_card_date   DATE NULL,

  -- money
  application_fee   JSON NULL,          -- by category
  salary_min        INT UNSIGNED NULL,
  salary_max        INT UNSIGNED NULL,

  -- links, verification
  official_pdf_url  VARCHAR(700) NULL,
  apply_url         VARCHAR(700) NULL,
  source_url        VARCHAR(700) NULL,
  source_id         BIGINT UNSIGNED NULL,      -- which scraper found it
  verified_by       BIGINT UNSIGNED NULL,      -- admin user
  verified_at       TIMESTAMP NULL,

  status            ENUM('draft','pending_review','published',
                        'expired','cancelled') DEFAULT 'draft',
  published_at      TIMESTAMP NULL,
  view_count        BIGINT UNSIGNED DEFAULT 0,
  save_count        INT UNSIGNED DEFAULT 0,

```

```

        created_at          TIMESTAMP NULL,
        updated_at          TIMESTAMP NULL,
        deleted_at          TIMESTAMP NULL,

        FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE SET NULL,
        INDEX idx_status_published (status, published_at DESC),
        INDEX idx_deadline (apply_end_date),
        INDEX idx_eligibility_match (min_qualification, max_age, status),
        INDEX idx_job_type (job_type, status, published_at DESC)
    );

CREATE TABLE notification_saves (
    user_id          BIGINT UNSIGNED NOT NULL,
    notification_id BIGINT UNSIGNED NOT NULL,
    remind_at        TIMESTAMP NULL,
    created_at        TIMESTAMP NULL,
    PRIMARY KEY (user_id, notification_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (notification_id) REFERENCES notifications(id) ON DELETE CASCADE,
    INDEX idx_remind (remind_at)
);

```

Quiz and streaks

```

CREATE TABLE questions (
    id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    exam_id     BIGINT UNSIGNED NULL,
    subject     VARCHAR(80) NULL,
    topic       VARCHAR(120) NULL,
    difficulty   ENUM('easy','medium','hard') DEFAULT 'medium',
    question_type ENUM('mcq','multi','numeric') DEFAULT 'mcq',
    question     JSON NOT NULL,          -- {"en": "...", "te": "..."}
    options      JSON NOT NULL,          -- {"en":[...], "te":[...]}
    correct_index TINYINT NOT NULL,
    explanation  JSON NULL,
    source_year  SMALLINT NULL,          -- previous-year tagging
    is_current_affairs TINYINT(1) DEFAULT 0,
    times_served INT UNSIGNED DEFAULT 0,
    times_correct INT UNSIGNED DEFAULT 0,
    created_at    TIMESTAMP NULL,
    updated_at    TIMESTAMP NULL,
    deleted_at    TIMESTAMP NULL,
    INDEX idx_subject_topic (subject, topic),
    INDEX idx_selection (is_current_affairs, difficulty, times_served)
);

CREATE TABLE daily_quizzes (
    id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    quiz_date    DATE UNIQUE NOT NULL,
    question_ids JSON NOT NULL,
    published_at TIMESTAMP NULL,
    attempt_count INT UNSIGNED DEFAULT 0,
    created_at    TIMESTAMP NULL,
    updated_at    TIMESTAMP NULL,
    INDEX idx_date (quiz_date DESC)
);

CREATE TABLE quiz_attempts (
    id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    user_id     BIGINT UNSIGNED NOT NULL,
    daily_quiz_id BIGINT UNSIGNED NULL,
    test_id     BIGINT UNSIGNED NULL,
    answers      JSON NOT NULL,          -- [{"q":1,"a":2,"t":14}]  t = seconds

```

```

    score            DECIMAL(6,2) NOT NULL,
    total_marks      DECIMAL(6,2) NOT NULL,
    time_taken_sec   INT UNSIGNED NULL,
    completed_at     TIMESTAMP NULL,
    created_at       TIMESTAMP NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_date (user_id, created_at DESC),
    INDEX idx_quiz_score (daily_quiz_id, score DESC)
);

CREATE TABLE streaks (
    user_id          BIGINT UNSIGNED PRIMARY KEY,
    current_streak   INT UNSIGNED DEFAULT 0,
    longest_streak   INT UNSIGNED DEFAULT 0,
    last_active_date DATE NULL,
    freezes_left     TINYINT UNSIGNED DEFAULT 1,
    freeze_reset_at  DATE NULL,
    updated_at       TIMESTAMP NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_current (current_streak DESC)
);

```

Material library

```

CREATE TABLE materials (
    id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    uuid              CHAR(36) UNIQUE NOT NULL,
    exam_id           BIGINT UNSIGNED NULL,
    uploaded_by       BIGINT UNSIGNED NULL,          -- NULL = official
    slug              VARCHAR(200) UNIQUE NOT NULL,
    title             JSON NOT NULL,
    description        JSON NULL,
    subject           VARCHAR(80) NULL,
    topic             VARCHAR(120) NULL,
    locale            CHAR(5) NOT NULL,              -- material's own language
    file_path          VARCHAR(500) NULL,             -- R2 key
    file_size_kb       INT UNSIGNED NULL,
    file_type          ENUM('pdf','html','image') DEFAULT 'pdf',
    html_content       LONGTEXT NULL,                 -- mobile-readable version
    source_type        ENUM('official','original','user_notes') NOT NULL,
    source_url         VARCHAR(700) NULL,
    copyright_confirmed TINYINT(1) DEFAULT 0,         -- upload warranty checkbox
    status             ENUM('pending_review','published','rejected')
                     DEFAULT 'pending_review',
    reviewed_by        BIGINT UNSIGNED NULL,
    reviewed_at        TIMESTAMP NULL,
    rejection_reason   VARCHAR(400) NULL,
    download_count     INT UNSIGNED DEFAULT 0,
    created_at         TIMESTAMP NULL,
    updated_at         TIMESTAMP NULL,
    deleted_at         TIMESTAMP NULL,
    INDEX idx_status_exam (status, exam_id),
    INDEX idx_downloads (download_count DESC)
);

```

Community

```

CREATE TABLE posts (
    id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    uuid              CHAR(36) UNIQUE NOT NULL,
    user_id           BIGINT UNSIGNED NOT NULL,

```

```

    exam_id          BIGINT UNSIGNED NULL,
    slug             VARCHAR(250) UNIQUE NOT NULL,
    title            VARCHAR(300) NOT NULL,
    body             LONGTEXT NOT NULL,
    source_locale    CHAR(5) NOT NULL,
    subject          VARCHAR(80) NULL,
    image_path       VARCHAR(500) NULL,
    audio_path       VARCHAR(500) NULL,          -- voice-note doubts
    upvotes          INT DEFAULT 0,
    answer_count     INT UNSIGNED DEFAULT 0,
    best_answer_id   BIGINT UNSIGNED NULL,
    view_count       INT UNSIGNED DEFAULT 0,
    status           ENUM('published','flagged','removed') DEFAULT 'published',
    created_at       TIMESTAMP NULL,
    updated_at       TIMESTAMP NULL,
    deleted_at       TIMESTAMP NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_exam_recent (exam_id, created_at DESC),
    INDEX idx_unanswered (answer_count, created_at DESC),
    FULLTEXT KEY ft_search (title, body)
);

CREATE TABLE answers (
    id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    post_id           BIGINT UNSIGNED NOT NULL,
    user_id           BIGINT UNSIGNED NOT NULL,
    body             LONGTEXT NOT NULL,
    source_locale    CHAR(5) NOT NULL,
    image_path       VARCHAR(500) NULL,
    upvotes          INT DEFAULT 0,
    is_best           TINYINT(1) DEFAULT 0,
    status           ENUM('published','flagged','removed') DEFAULT 'published',
    created_at       TIMESTAMP NULL,
    updated_at       TIMESTAMP NULL,
    deleted_at       TIMESTAMP NULL,
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_post_votes (post_id, upvotes DESC)
);

CREATE TABLE content_translations (
    id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    translatable_type VARCHAR(60) NOT NULL,    -- 'post' | 'answer'
    translatable_id   BIGINT UNSIGNED NOT NULL,
    locale            CHAR(5) NOT NULL,
    body             LONGTEXT NOT NULL,
    engine            ENUM('ai','human') DEFAULT 'ai',
    created_at       TIMESTAMP NULL,
    UNIQUE KEY uk_translation (translatable_type, translatable_id, locale)
);

CREATE TABLE votes (
    user_id           BIGINT UNSIGNED NOT NULL,
    votable_type      VARCHAR(60) NOT NULL,
    votable_id        BIGINT UNSIGNED NOT NULL,
    value             TINYINT NOT NULL,        -- +1 / -1
    created_at       TIMESTAMP NULL,
    PRIMARY KEY (user_id, votable_type, votable_id)
);

```

Advertising

```
CREATE TABLE ad_campaigns (
  id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  advertiser_name   VARCHAR(160) NOT NULL,
  contact_phone     VARCHAR(15) NULL,
  creative_path     VARCHAR(500) NULL,
  headline          JSON NULL,
  target_url        VARCHAR(700) NOT NULL,
  placement         ENUM('feed', 'exam_page', 'pre_download', 'quiz_sponsor')
                  NOT NULL,
  target_districts  JSON NULL,
  target_exams      JSON NULL,
  target_locales    JSON NULL,
  starts_at         DATE NOT NULL,
  ends_at           DATE NOT NULL,
  daily_cap         INT UNSIGNED NULL,
  amount_paid       DECIMAL(10,2) NULL,
  status            ENUM('draft', 'active', 'paused', 'completed') DEFAULT 'draft',
  created_at        TIMESTAMP NULL,
  updated_at        TIMESTAMP NULL,
  INDEX idx_active (status, starts_at, ends_at)
);

CREATE TABLE ad_events (
  id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  campaign_id       BIGINT UNSIGNED NOT NULL,
  event_type        ENUM('impression', 'click') NOT NULL,
  user_id           BIGINT UNSIGNED NULL,
  district          VARCHAR(60) NULL,
  occurred_at       TIMESTAMP NOT NULL,
  INDEX idx_campaign_time (campaign_id, occurred_at)
) PARTITION BY RANGE (UNIX_TIMESTAMP(occurred_at)) (
  -- monthly partitions, added by a scheduled command
);
```

Note on `ad_events` : this table grows fastest of anything in the system. Write raw events, roll up hourly into a summary table, and drop partitions older than 90 days. Never query the raw table for a dashboard.

Supporting tables

```
CREATE TABLE scrape_sources (
  id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  name              VARCHAR(120) NOT NULL,
  url               VARCHAR(700) NOT NULL,
  parser_class      VARCHAR(160) NOT NULL,
  frequency_min     INT UNSIGNED DEFAULT 30,
  last_run_at       TIMESTAMP NULL,
  last_success_at   TIMESTAMP NULL,
  consecutive_failures INT UNSIGNED DEFAULT 0,
  is_active         TINYINT(1) DEFAULT 1
);

CREATE TABLE translation_queue (
  id                BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  model_type        VARCHAR(60) NOT NULL,
  model_id          BIGINT UNSIGNED NOT NULL,
  field             VARCHAR(60) NOT NULL,
  source_locale     CHAR(5) NOT NULL,
```

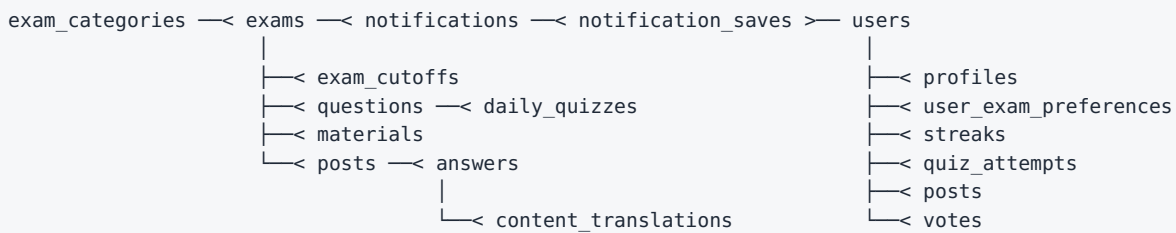
```

target_locale CHAR(5) NOT NULL,
source_text   TEXT NOT NULL,
translated_text TEXT NULL,
status        ENUM('pending','translated','approved','rejected')
              DEFAULT 'pending',
is_critical   TINYINT(1) DEFAULT 0,    -- dates/eligibility → human required
reviewed_by   BIGINT UNSIGNED NULL,
created_at    TIMESTAMP NULL,
updated_at    TIMESTAMP NULL,
INDEX idx_status (status, is_critical)
);

CREATE TABLE push_tokens (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NOT NULL,
  token       VARCHAR(500) NOT NULL,
  platform    ENUM('web','android','ios') NOT NULL,
  is_active   TINYINT(1) DEFAULT 1,
  created_at  TIMESTAMP NULL,
  UNIQUE KEY uk_token (token(191)),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

```

4.3 Entity relationships



Chapter 5 — Multi-Language Implementation

సారాంశం: మూడు పొరలు, మూడు వేర్వేరు పద్ధతులు. అన్నిటికీ ఒకటే వాడితే ప్రాజెక్ట్ ఆగిపోతుంది.

5.1 Configuration

```
// config/locales.php
return [
    'supported' => [
        'te' => ['name' => 'తెలుగు', 'native' => 'తెలుగు', 'dir' => 'ltr', 'active' => true],
        'en' => ['name' => 'English', 'native' => 'English', 'dir' => 'ltr', 'active' => true],
        'hi' => ['name' => 'Hindi', 'native' => 'हिन्दी', 'dir' => 'ltr', 'active' => false],
        'ta' => ['name' => 'Tamil', 'native' => 'தமிழ்', 'dir' => 'ltr', 'active' => false],
    ],
    'default' => 'te',
    'fallback' => 'en',
];
```

Adding a language later is a config flip plus content, not a code change. That is the whole point of doing this properly at the start.

5.2 Routing

```
// routes/web.php
Route::redirect('/', '/te'); // or geo/cookie-aware, see middleware

Route::group([
    'prefix' => '{locale}',
    'where' => ['locale' => 'te|en|hi|ta'],
    'middleware' => ['setlocale'],
], function () {
    Route::get('/', [HomeController::class, 'index'])->name('home');

    Route::get('/notifications', [NotificationController::class, 'index'])->
>name('notifications.index');
    Route::get('/notifications/{slug}', [NotificationController::class, 'show'])->
>name('notifications.show');

    Route::get('/exams', [ExamController::class, 'index'])->name('exams.index');
    Route::get('/exams/{slug}', [ExamController::class, 'show'])->name('exams.show');

    Route::get('/quiz', [QuizController::class, 'today'])->name('quiz.today');
    Route::get('/community', [CommunityController::class, 'index'])->name('community.index');
    Route::get('/community/{slug}', [CommunityController::class, 'show'])->name('community.show');
    Route::get('/material', [MaterialController::class, 'index'])->name('material.index');

    Route::middleware('auth')->group(function () {
        Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
```

```
Route::get('/saved', [SavedController::class, 'index']->name('saved'));
});
});
```

5.3 The locale middleware

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Support\Facades\App;
use Illuminate\Support\Facades\Cookie;
use Illuminate\Support\Facades\URL;

class SetLocale
{
    public function handle($request, Closure $next)
    {
        $locale = $request->route('locale');

        $supported = collect(config('locales.supported'))
            ->filter(fn ($l) => $l['active'])->keys()->all();

        if (! in_array($locale, $supported, true)) {
            $locale = config('locales.default');
        }

        App::setLocale($locale);

        // every route() call inherits the current locale – no manual passing
        URL::defaults(['locale' => $locale]);

        Cookie::queue('locale', $locale, 60 * 24 * 365);

        // keep the logged-in user's preference in sync
        if ($user = $request->user()) {
            if ($user->preferred_locale !== $locale) {
                $user->updateQuietly(['preferred_locale' => $locale]);
            }
        }

        return $next($request);
    }
}
```

5.4 Layer 1 — UI strings

Use JSON files keyed by the English sentence, not PHP arrays keyed by dot-notation. A missing key then falls back to readable English instead of showing `messages.feed.empty_state`.

```
// lang/te.json
{
    "Download PDF": "PDF డౌన్‌లోడ్ చేయండి",
    "You are eligible": "మీరు అర్హులు",
    "Not eligible": "అర్హత లేదు",
    "Post your doubt": "మీ సందేహాన్ని పోస్ట్ చేయండి",
    "Last date": "చివరి తేదీ",
```

```
"Day :count streak": ":count రోజుల స్ట్రీక్",
"vacancies": "ఖాళీలు"
}
```

```
{{ __('Download PDF') }}
{{ __('Day :count streak', ['count' => $streak]) }}
```

5.5 Layer 2 — Editorial content

```
composer require spatie/laravel-translatable
```

```
namespace App\Models;

use Spatie\Translatable\HasTranslations;

class ExamNotification extends Model
{
    use HasTranslations;

    public array $translatable = [
        'title', 'description', 'qualification_notes',
        'meta_title', 'meta_description',
    ];

    protected $casts = [
        'vacancy_breakdown' => 'array',
        'age_relaxation' => 'array',
        'allowed_states' => 'array',
        'application_fee' => 'array',
        'apply_end_date' => 'date',
        'age_reference_date' => 'date',
    ];
}
```

```
$n->title; // current locale, auto-fallback
$n->getTranslation('title', 'en');
$n->setTranslation('title', 'te', 'పోలీస్ కానిస్టేబుల్ నోటిఫికేషన్ 2026');
```

Filament field for the admin panel:

```
use Filament\Forms\Components\Tabs;

Tabs::make('Translations')->tabs(
    collect(config('locales.supported'))
    ->filter(fn ($l) => $l['active'])
    ->map(fn ($cfg, $code) => Tabs\Tab::make($cfg['native'])->schema([
        TextInput::make("title.$code")->required($code === 'en'),
        RichEditor::make("description.$code"),
    ])->values()->all()
);
```

5.6 Layer 3 — User content, translated lazily

```
namespace App\Services;

class UserContentTranslator
{
    public function translate(Model $model, string $targetLocale): string
    {
        if ($model->source_locale === $targetLocale) {
            return $model->body;
        }

        $cached = ContentTranslation::where([
            'translatable_type' => $model->getMorphClass(),
            'translatable_id'   => $model->id,
            'locale'            => $targetLocale,
        ])->first();

        if ($cached) {
            return $cached->body;
        }

        $translated = app(AiTranslator::class)->translate(
            text: $model->body,
            from: $model->source_locale,
            to:   $targetLocale,
            glossary: Glossary::forLocale($targetLocale),
        );

        ContentTranslation::create([
            'translatable_type' => $model->getMorphClass(),
            'translatable_id'   => $model->id,
            'locale'            => $targetLocale,
            'body'              => $translated,
            'engine'            => 'ai',
        ]);

        return $translated;
    }
}
```

Translate on demand, never in bulk. Ninety percent of community posts will never be read in another language; pre-translating them all would burn budget for nothing.

Always label machine translation in the UI: "యంత్ర అనువాదం — *Machine translated*".

5.7 The glossary — do not skip this

Certain terms must never be translated: **Group 2** stays గ్రూప్ 2, not a semantic translation. **Constable**, **SSC**, **Tahsildar** all have established Telugu forms that aspirants search for.

```
// database/seeder/GlossarySeeder.php
[
    'Group 2'      => ['te' => 'గ్రూప్ 2'],
    'Constable'    => ['te' => 'కాన్స్టేబుల్'],
    'Sub Inspector' => ['te' => 'సబ్ ఇన్స్పెక్టర్'],
]
```

```
'Naib Tahsildar' => ['te' => 'నాయబ్ తహసీల్దార్'],
'Notification'   => ['te' => 'నోటిఫికేషన్'], // NOT 'ప్రకటన'
]
```

The glossary is injected into every AI translation prompt. Getting these words right is the difference between ranking on Google and not, because these are the words people actually type.

5.8 Cache keys must include locale

```
// WRONG — the most common i18n bug there is
Cache::remember("exam:${slug}", 3600, fn () => $this->build($slug));

// RIGHT
Cache::remember("exam:${slug}:" . app()->getLocale(), 3600, fn () => $this->build($slug));
```

Make it impossible to get wrong:

```
// app/Support/Locale.php
public static function cacheKey(string $key): string
{
    return $key . ':' . app()->getLocale();
}
```

Then lint for raw `Cache::remember(` calls in review.

5.9 Typography

```
@font-face {
    font-family: 'Noto Sans Telugu';
    src: url('/fonts/noto-sans-telugu-subset.woff2') format('woff2');
    font-display: swap;
    unicode-range: U+0C00-0C7F, U+200C-200D;
}

html[lang="te"] {
    font-family: 'Noto Sans Telugu', system-ui, sans-serif;
    line-height: 1.85; /* Telugu glyphs are taller */
    letter-spacing: 0;
}

html[lang="en"] {
    font-family: system-ui, -apple-system, sans-serif;
    line-height: 1.6;
}
```

Rules: - Self-host the font. Google Fonts adds a DNS lookup and a third-party dependency for a font we serve on every page. - Subset it. Full Noto Sans Telugu is 300 KB+; a subset is around 60 KB. - Keep numerals in Latin script. 2026 , ₹28,940 , 783 ఖాళీలు . Telugu numerals are not read fluently by this audience. - Telugu needs more vertical space. Test every component with Telugu text, not English — Telugu strings run 15–30% longer and will break layouts designed against English.

Chapter 6 — Module: Notification Feed

సారాంశం: ఇదే ఉత్పత్తి యొక్క గుండె. అర్హత తనిఖీయే మన ప్రత్యేకత.

6.1 The eligibility engine

This is the single highest-leverage piece of code in the product. It converts a notice board into an advisor.

```
namespace App\Services;

use App\Models\ExamNotification;
use App\Models\Profile;
use Carbon\Carbon;

class EligibilityService
{
    /**
     * @return array{status:string, matched:array, failed:array, score:float}
     */
    public function check(ExamNotification $n, ?Profile $p): array
    {
        if (!$p) {
            return ['status' => 'unknown', 'matched' => [], 'failed' => [], 'score' => 0];
        }

        $matched = [];
        $failed = [];

        // --- qualification ---
        if ($n->min_qualification) {
            $required = $this->rank($n->min_qualification);
            $has = $this->rank($p->highest_qualification);

            $has >= $required
                ? $matched[] = ['key' => 'qualification', 'label' => __('Qualification')]
                : $failed[] = [
                    'key' => 'qualification',
                    'label' => __('Qualification'),
                    'reason' => __('Requires :q', ['q' => __($n->min_qualification)]),
                ];
        }

        // --- age, with category relaxation ---
        if ($p->date_of_birth && ($n->min_age || $n->max_age)) {
            $refDate = $n->age_reference_date ?? $n->apply_end_date ?? now();
            $age = Carbon::parse($p->date_of_birth)->diffInYears($refDate);

            $relaxation = 0;
            $rules = $n->age_relaxation ?? [];
            $relaxation += $rules[$p->category] ?? 0;
            if ($p->is_pwd) $relaxation += $rules['pwd'] ?? 0;
            if ($p->is_ex_serviceman) $relaxation += $rules['ex_serviceman'] ?? 0;

            $effectiveMax = $n->max_age ? $n->max_age + $relaxation : null;
        }
    }
}
```

```

    $ageOk = (! $n->min_age || $age >= $n->min_age)
        && (! $effectiveMax || $age <= $effectiveMax);

    $ageOk
    ? $matched[] = ['key' => 'age', 'label' => __('Age')]
    : $failed[] = [
        'key' => 'age',
        'label' => __('Age'),
        'reason' => __('Age limit :min-:max', [
            'min' => $n->min_age ?? 18,
            'max' => $effectiveMax ?? '-',
        ]),
    ];
}

// --- domicile ---
if (! empty($n->allowed_states)) {
    in_array($p->state, $n->allowed_states, true)
    ? $matched[] = ['key' => 'state', 'label' => __('State')]
    : $failed[] = [
        'key' => 'state',
        'label' => __('State'),
        'reason' => __('Only for :states', ['states' => implode(', ', $n->allowed_states)]),
    ];
}

// --- gender ---
if ($n->gender_restriction !== 'any' && $p->gender) {
    $n->gender_restriction === $p->gender
    ? $matched[] = ['key' => 'gender', 'label' => __('Gender')]
    : $failed[] = ['key' => 'gender', 'label' => __('Gender'), 'reason' => __('Not
applicable')];
}

$total = count($matched) + count($failed);
$score = $total > 0 ? count($matched) / $total : 0;

return [
    'status' => empty($failed) ? 'eligible' : ($score >= 0.5 ? 'partial' : 'not_eligible'),
    'matched' => $matched,
    'failed' => $failed,
    'score' => $score,
];
}

private function rank(?string $q): int
{
    return [
        '10th' => 1, '12th' => 2, 'iti' => 2, 'diploma' => 3,
        'degree' => 4, 'btech' => 4, 'mbbs' => 5, 'pg' => 5, 'phd' => 6,
   ][$q] ?? 0;
}
}

```

Design note: returning **partial** rather than a binary yes/no is deliberate. A user who fails only on age still wants to see the notification — they may have a relaxation we do not know about, or a family member who qualifies. Never hide content; label it.

Legal note: always render a caveat next to the badge — *"Please verify against the official notification before applying."* An automated eligibility check is guidance, not a legal determination.

6.2 Personalised feed query

```
class FeedService
{
    public function forUser(?User $user, array $filters = [], int $perPage = 20)
    {
        $q = ExamNotification::query()
            ->where('status', 'published')
            ->where(fn ($q) => $q->whereNull('apply_end_date')
                ->orWhere('apply_end_date', '>=', today()))
            ->with('exam:id,slug,name');

        if ($user && $user->examPreferences->isNotEmpty()) {
            $ids = $user->examPreferences->pluck('id');
            // preferred exams float to the top, everything else still visible
            $q->orderByRaw('CASE WHEN exam_id IN (' . $ids->implode(',') . ') THEN 0 ELSE 1 END');
        }

        foreach (['job_type', 'min_qualification'] as $f) {
            if (! empty($filters[$f])) $q->where($f, $filters[$f]);
        }

        if (! empty($filters['district'])) {
            $q->where(fn ($q) => $q->whereNull('allowed_districts')
                ->orWhereJsonContains('allowed_districts', $filters['district']));
        }

        if (! empty($filters['closing_soon'])) {
            $q->whereBetween('apply_end_date', [today(), today()->addDays(7)]);
        }

        return $q->orderByDesc('published_at')->paginate($perPage);
    }
}
```

6.3 Feed component

```
namespace App\Livewire\Feed;

use Livewire\Component;
use Livewire\WithPagination;

class NotificationFeed extends Component
{
    use WithPagination;

    public array $filters = [];
    public bool $onlyEligible = false;

    protected $queryString = ['filters', 'onlyEligible'];

    public function render()
    {
        {
            $notifications = app(FeedService::class)
                ->forUser(auth()->user(), $this->filters);

            $profile = auth()->user()?->profile;
            $checker = app(EligibilityService::class);

            $decorated = $notifications->through(fn ($n) => tap($n, function ($n) use ($checker, $profile) {
                $n->eligibility = $checker->check($n, $profile);
            }));
        }
    }
}
```

```

    }));

    if ($this->onlyEligible) {
        $decorated = $decorated->filter(fn ($n) => $n->eligibility['status'] === 'eligible');
    }

    return view('livewire.feed.notification-feed', [
        'notifications' => $decorated,
        'ad'            => app(AdService::class)->forPlacement('feed'),
    ]);
}
}

```

6.4 Notification card

```

<article class="border-b border-neutral-200 py-4">
    <div class="flex items-start justify-between gap-3">
        <div class="min-w-0">
            <h3 class="font-semibold text-base leading-snug">
                <a href="{{ route('notifications.show', ['slug' => $n->slug]) }}"
                    class="hover:underline">{{ $n->title }}</a>
            </h3>
            <p class="mt-1 text-sm text-neutral-600">{{ $n->organisation }}</p>
        </div>

        @if($n->eligibility['status'] === 'eligible')
            <span class="shrink-0 rounded-full bg-green-100 px-2.5 py-1 text-xs font-medium text-green-800">
                ✓ {{ __('You are eligible') }}
            </span>
        @elseif($n->eligibility['status'] === 'partial')
            <span class="shrink-0 rounded-full bg-amber-100 px-2.5 py-1 text-xs font-medium text-amber-800">
                {{ __(':n of :t match', [
                    'n' => count($n->eligibility['matched']),
                    't' => count($n->eligibility['matched']) + count($n->eligibility['failed']),
                ]) }}
            </span>
        @endif
    </div>

    <dl class="mt-3 flex flex-wrap gap-x-5 gap-y-1 text-sm">
        @if($n->total_vacancies)
            <div><dt class="inline text-neutral-500">{{ __('vacancies') }}:</dt>
                <dd class="inline font-medium">{{ number_format($n->total_vacancies) }}</dd></div>
        @endif
        @if($n->apply_end_date)
            <div><dt class="inline text-neutral-500">{{ __('Last date') }}:</dt>
                <dd class="inline font-medium {{ $n->apply_end_date->diffInDays() <= 3 ? 'text-red-600' : '' }}">
                    {{ $n->apply_end_date->format('d M Y') }}</dd></div>
        @endif
    </dl>

    <div class="mt-3 flex gap-2">
        <a href="{{ route('notifications.show', ['slug' => $n->slug]) }}"
            class="rounded-lg bg-neutral-900 px-3 py-1.5 text-sm text-white">{{ __('Details') }}</a>
        <button wire:click="save({{ $n->id }})"
            class="rounded-lg border px-3 py-1.5 text-sm">{{ __('Save') }}</button>
    </div>
</article>

```


Chapter 7 — Module: Exam Hub (the SEO engine)

సారాంశం: ఈ పేజీలే మనకు ఉచిత ట్రాఫిక్ తెస్తాయి. వీటిని ఉత్పత్తిలా చూడాలి.

7.1 Controller with aggressive caching

```
class ExamController extends Controller
{
  public function show(string $locale, string $slug)
  {
    $exam = Cache::remember(
      Locale::cacheKey("exam:$slug"),
      now()->addHours(6),
      fn () => Exam::where('slug', $slug)
        ->with([
          'category',
          'cutoffs' => fn ($q) => $q->orderByDesc('year')->limit(15),
          'notifications' => fn ($q) => $q->where('status', 'published')
            ->latest('published_at')->limit(5),
          'materials' => fn ($q) => $q->where('status', 'published')
            ->orderByDesc('download_count')->limit(10),
        ])
        ->firstOrFail()
    );

    // fire-and-forget view counter; never block the response
    dispatch(fn () => $exam->incrementQuietly('view_count'))->afterResponse();

    return view('exam.show', [
      'exam' => $exam,
      'seo' => SeoBuilder::forExam($exam),
      'schema' => $this->structuredData($exam),
    ]);
  }

  private function structuredData(Exam $exam): array
  {
    return [
      '@context' => 'https://schema.org',
      '@type' => 'Course',
      'name' => $exam->name,
      'description' => Str::limit(strip_tags($exam->description ?? ''), 300),
      'provider' => ['@type' => 'Organization', 'name' => $exam->conducting_body],
      'inLanguage' => app()->getLocale(),
    ];
  }
}
```

7.2 Page structure — fixed, in this order

Do not let this vary page to page. Consistency is what teaches Google what the template means.

```

@extends('layouts.app')

@section('content')
<article>
    <h1>{{ $exam->name }} {{ now()->year }}</h1>
    <p class="lead">{{ $exam->description }}</p>

    @include('exam.partials.latest-notification')    {{-- 1. status now --}}
    @include('exam.partials.quick-facts')           {{-- 2. table: body, vacancies, dates --}}
    @include('exam.partials.eligibility')           {{-- 3. --}}
    @include('exam.partials.exam-pattern')          {{-- 4. --}}
    @include('exam.partials.syllabus')              {{-- 5. expandable + PDF --}}
    @include('exam.partials.previous-papers')       {{-- 6. --}}
    @include('exam.partials.cutoffs')               {{-- 7. table by year/category --}}
    @include('exam.partials.free-material')         {{-- 8. --}}
    @include('exam.partials.community-questions')   {{-- 9. top doubts for this exam --}}
    @include('exam.partials.related-exams')         {{-- 10. --}}
    @include('exam.partials.faq')                   {{-- 11. schema.org FAQPage --}}
</article>
@endsection

```

Section 9 is worth calling out: surfacing community questions on the exam page makes every answered doubt do double duty — it helps the reader and it adds unique, long-tail indexable text to a page that would otherwise be static.

7.3 SEO head

```

{{-- resources/views/layouts/partials/seo.blade.php --}}
<title>{{ $seo['title'] }}</title>
<meta name="description" content="{{ $seo['description'] }}">
<link rel="canonical" href="{{ $seo['canonical'] }}">

@foreach($seo['alternates'] as $code => $url)
    <link rel="alternate" hreflang="{{ $code }}-IN" href="{{ $url }}">
@endforeach
<link rel="alternate" hreflang="x-default" href="{{ $seo['alternates']['en'] }}">

<meta property="og:title" content="{{ $seo['title'] }}">
<meta property="og:type" content="article">
<meta property="og:locale" content="{{ str_replace('-', '_', app()->getLocale()) }}_IN">

@isset($schema)
<script type="application/ld+json">@json($schema, JSON_UNESCAPED_UNICODE)</script>
@endisset

```

```

// app/Support/SeoBuilder.php
public static function forExam(Exam $exam): array
{
    $alternates = collect(config('locales.supported'))
        ->filter(fn ($l) => $l['active'])
        ->mapWithKeys(fn ($cfg, $code) => [
            $code => route('exams.show', ['locale' => $code, 'slug' => $exam->slug]),
        ])->all();

    return [
        'title' => $exam->meta_title ?: $exam->name . ' ' . now()->year,
        'description' => $exam->meta_description ?: Str::limit(strip_tags($exam->description), 155),
        'canonical' => route('exams.show', ['locale' => app()->getLocale(), 'slug' => $exam->slug]),
    ];
}

```

```

    'alternates' => $alternates,
  ];
}

```

7.4 JobPosting structured data for notifications

This is what gets our notifications into Google Jobs — a significant free traffic source that competitors in this space largely ignore.

```

[
  '@context' => 'https://schema.org',
  '@type'    => 'JobPosting',
  'title'    => $n->title,
  'description' => $n->description,
  'datePosted' => $n->published_at?->toIso8601String(),
  'validThrough' => $n->apply_end_date?->toIso8601String(),
  'employmentType' => 'FULL_TIME',
  'hiringOrganization' => ['@type' => 'Organization', 'name' => $n->organisation],
  'jobLocation' => [
    '@type' => 'Place',
    'address' => ['@type' => 'PostalAddress', 'addressRegion' => $n->allowed_states[0] ?? 'TS',
    'addressCountry' => 'IN'],
  ],
  'baseSalary' => $n->salary_min ? [
    '@type' => 'MonetaryAmount', 'currency' => 'INR',
    'value' => ['@type' => 'QuantitativeValue',
      'minValue' => $n->salary_min, 'maxValue' => $n->salary_max, 'unitText' => 'MONTH'],
  ] : null,
  'totalJobOpenings' => $n->total_vacancies,
]

```

7.5 Sitemaps

One sitemap per locale per content type, referenced from a sitemap index.

```

/sitemap.xml (index)
├─ /sitemap-exams-te.xml
├─ /sitemap-exams-en.xml
├─ /sitemap-notifications-te.xml
├─ /sitemap-notifications-en.xml
├─ /sitemap-community-te.xml
└─ /sitemap-material-te.xml

```

Regenerate nightly. Submit each locale separately in Google Search Console.

Chapter 8 — Module: Daily Quiz and Streaks

సారాంశం: ఇదే రిటెన్షన్ ఇంజిన్. ఇది పని చేయకపోతే మిగతా అంతా వృధా.

8.1 Automated daily publication

```
namespace App\Console\Commands;

class PublishDailyQuiz extends Command
{
    protected $signature = 'quiz:publish [--date=]';

    public function handle(): int
    {
        $date = $this->option('date') ? Carbon::parse($this->option('date')) : today();

        if (DailyQuiz::whereDate('quiz_date', $date)->exists()) {
            $this->warn('Already published for ' . $date->toDateString());
            return self::SUCCESS;
        }

        $questions = collect()
            ->merge($this->pick(currentAffairs: true, count: 5))
            ->merge($this->pick(currentAffairs: false, count: 3))
            ->merge($this->pickPreviousYear(2));

        if ($questions->count() < 10) {
            // never publish a short quiz – alert the content team instead
            report(new \RuntimeException('Insufficient questions for ' . $date->toDateString()));
            Notification::route('mail', config('app.ops_email'))
                ->notify(new QuizPoolLowNotification($questions->count()));
            return self::FAILURE;
        }

        $quiz = DailyQuiz::create([
            'quiz_date' => $date,
            'question_ids' => $questions->pluck('id')->all(),
            'published_at' => now(),
        ]);

        Question::whereIn('id', $quiz->question_ids)->increment('times_served');

        dispatch(new SendDailyQuizPush($quiz));

        return self::SUCCESS;
    }

    private function pick(bool $currentAffairs, int $count)
    {
        return Question::where('is_current_affairs', $currentAffairs)
            ->whereNotIn('id', $this->recentlyUsedIds())
            ->whereNotNull('question->te') // must exist in Telugu
            ->orderBy('times_served') // rotate the pool evenly
            ->inRandomOrder()
            ->limit($count)->get();
    }
}
```

```

    }

    private function recentlyUsedIds(): array
    {
        return DailyQuiz::where('quiz_date', '>=', today()->subDays(60))
            ->pluck('question_ids')->flatten()->unique()->all();
    }
}

```

```

// routes/console.php
Schedule::command('quiz:publish')->dailyAt('06:45')->timezone('Asia/Kolkata');
Schedule::job(new SendDailyQuizPush)->dailyAt('07:00')->timezone('Asia/Kolkata');

```

Note the 15-minute gap: publish first, push second. Never send a notification for content that has not finished writing.

8.2 Streak service

```

namespace App\Services;

class StreakService
{
    public function record(User $user): array
    {
        $streak = Streak::firstOrCreate(['user_id' => $user->id]);
        $today = today();
        $last = $streak->last_active_date;

        if ($last->isSameDay($today)) {
            return ['streak' => $streak->current_streak, 'changed' => false];
        }

        if ($last->isSameDay($today->copy()->subDay())) {
            $streak->current_streak++; // continued
        } elseif ($last && $last->diffInDays($today) === 2 && $streak->freezes_left > 0) {
            $streak->freezes_left--; // one grace day
            $streak->current_streak++;
        } else {
            $streak->current_streak = 1; // reset
        }

        $streak->longest_streak = max($streak->longest_streak, $streak->current_streak);
        $streak->last_active_date = $today;

        if ($streak->freeze_reset_at->lt($today->copy()->startOfMonth())) {
            $streak->freezes_left = 1;
            $streak->freeze_reset_at = $today->copy()->startOfMonth();
        }

        $streak->save();

        // Redis sorted set – leaderboard reads become O(log n)
        Redis::zadd('leaderboard:streak:' . $today->format('Y-W'),
            $streak->current_streak, $user->id);

        if (in_array($streak->current_streak, [3, 7, 30, 100], true)) {
            event(new StreakMilestoneReached($user, $streak->current_streak));
        }
    }
}

```

```
        return ['streak' => $streak->current_streak, 'changed' => true];
    }
}
```

Why the freeze exists: users have exams, travel, and family emergencies. A streak that punishes real life gets abandoned, and once abandoned it never restarts. One free miss a month costs nothing and saves a meaningful share of users.

8.3 Shareable result — the viral loop

After completing the quiz, generate a WhatsApp-status-shaped image (1080×1920) server-side with the score, streak, and a short branded URL. One tap to share.

```
class QuizResultImage
{
    public function generate(QuizAttempt $attempt): string
    {
        $img = Image::canvas(1080, 1920, '#0F172A');
        // score, streak count, date, sadhana.study – large Telugu type
        $path = "share/{$attempt->id}.jpg";
        Storage::disk('r2')->put($path, (string) $img->encode('jpg', 85));
        return Storage::disk('r2')->url($path);
    }
}
```

This single feature is the cheapest acquisition channel in the product. Design it carefully.

Chapter 9 — Module: Material Library

సారాంశం: కాపీరైట్ నియమాలు కఠినంగా అమలు చేయాలి. ఇక్కడ రాజీ పడితే ప్రాజెక్ట్ మూసుకోవాలి.

9.1 Upload flow

```

User selects file
↓
Client validates: PDF or image, under 25 MB
↓
Copyright warranty checkbox – mandatory, logged with timestamp and IP
↓
Upload to R2 (quarantine prefix, not publicly readable)
↓
Job: extract text, generate thumbnail, scan for foreign watermarks/logos
↓
Status: pending_review
↓
Admin reviews in Filament – approve / reject with reason
↓
On approve: move to public prefix, index in Meilisearch, credit uploader

```

```

class MaterialUploadRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'file' => ['required', 'file', 'mimes:pdf,jpg,png', 'max:25600'],
            'title' => ['required', 'string', 'max:200'],
            'exam_id' => ['required', 'exists:exams,id'],
            'subject' => ['nullable', 'string', 'max:80'],
            'locale' => ['required', Rule::in(array_keys(config('locales.supported')))],
            'copyright_confirmed' => ['required', 'accepted'],
        ];
    }

    public function messages(): array
    {
        return [
            'copyright_confirmed.accepted' =>
                __('You must confirm this is your own work.'),
        ];
    }
}

```

9.2 Moderation queue rules

Reject on sight if the file shows: - Another brand's watermark or logo - Scanned book pages with a publisher's imprint, ISBN, or copyright page - Printed typeset text that is clearly from a commercial publication - Any material already flagged in a previous rejection

Accept: - Handwritten notes - Self-typed summaries - Government PDFs (uploaded by staff, marked `source_type = official`)

Never automate approval. At 500 uploads a day this is roughly two hours of one person's time, and it is the cheapest legal insurance available.

9.3 Mobile-readable versions

A 40 MB scanned PDF is unusable to our primary persona. For every approved PDF, generate an HTML version:

```
class GenerateReadableVersion implements ShouldQueue
{
    public function handle(Material $material): void
    {
        $text = (new \Smalot\PdfParser\Parser())
            ->parseFile(Storage::disk('r2')->path($material->file_path))
            ->getText();

        $material->update([
            'html_content' => (new HtmlFormatter)->fromPlainText($text),
        ]);
    }
}
```

Serve the HTML by default with a "Download PDF" option. Faster, searchable, indexable by Google, and readable on a small screen.

Chapter 10 — Module: Community

సారాంశం: అడిగే ముందు వెతకమని చెప్పాలి. అదే డూప్లికేట్ ప్రశ్నలను ఆపుతుంది.

10.1 Search-before-ask

As the user types a doubt title, query Meilisearch live and surface similar existing questions. This prevents duplicates, gives instant answers, and — critically — keeps the archive clean enough to rank on Google.

```
class AskQuestion extends Component
{
    public string $title = '';
    public array $similar = [];

    public function updatedTitle(): void
    {
        if (strlen($this->title) < 12) {
            $this->similar = [];
            return;
        }

        $this->similar = Post::search($this->title)
            ->where('status', 'published')
            ->take(4)->get()
            ->map(fn ($p) => [
                'title' => $p->title,
                'url' => route('community.show', ['slug' => $p->slug]),
                'answers' => $p->answer_count,
            ])->all();
    }
}
```

10.2 Reputation

Action	Points
Answer upvoted	+10
Answer marked best	+25
Question upvoted	+5
Answer downvoted	-2
Post removed by moderator	-20
Material approved	+15

Privilege	Threshold
Downvote	50
Edit tags on others' posts	200
Flag for moderation	300
Edit others' posts	800
Moderator tools	1500

Verified selected candidates get a permanent badge — this is the single strongest trust signal in the community and worth a manual verification process.

10.3 Moderation ladder

```
User flags → queue
↓
Auto-hide if 3+ flags from users with 300+ reputation
↓
Moderator reviews within 4 hours
↓
Actions: dismiss / edit / remove / warn / suspend / ban
```

Zero tolerance, immediate permanent ban: exam paper leaks, impersonation of officials, selling pirated material, personal attacks, phone numbers posted for "guaranteed job" scams. The last one is common in this space and does real financial harm to users — treat it as seriously as piracy.

Chapter 11 — Ingestion Pipeline

సారాంశం: ప్రభుత్వ సైట్ల నుంచి డేటా తీసుకోవడం. కానీ మనిషి తనిఖీ తప్పనిసరి.

11.1 Architecture

```

Scheduler (every 30 min)
↓
For each active source → dispatch ScrapeSource job
↓
Fetch listing page → diff against last seen
↓
New item found → fetch detail page + PDF
↓
Parse: structured extraction (rules first, AI fallback)
↓
Create notification with status = pending_review
↓
Alert content team (Slack/WhatsApp)
↓
HUMAN VERIFIES dates, eligibility, fees
↓
status = published → dispatch push + index + translate

```

11.2 Parser pattern

```

abstract class BaseScraper
{
    abstract public function fetchListing(): Collection;
    abstract public function parseDetail(string $html, string $url): array;

    public function run(ScrapeSource $source): void
    {
        try {
            foreach ($this->fetchListing() as $item) {
                if (ExamNotification::where('source_url', $item['url'])->exists()) {
                    continue;
                }

                $html = Http::timeout(30)
                    ->withUserAgent('SadhanaBot/1.0 (+https://sadhana.study/bot)')
                    ->get($item['url'])->body();

                $data = $this->parseDetail($html, $item['url']);

                ExamNotification::create([
                    ...$data,
                    'source_id' => $source->id,
                    'source_url' => $item['url'],
                    'status' => 'pending_review', // never auto-publish
                ]);

                event(new NotificationScraped($item['url']));
            }
        }
    }
}

```

```

    }

    $source->update(['last_success_at' => now(), 'consecutive_failures' => 0]);
} catch (\Throwable $e) {
    $source->increment('consecutive_failures');
    report($e);

    if ($source->consecutive_failures >= 3) {
        // government sites change layout without warning — fail loudly
        Notification::route('mail', config('app.ops_email'))
            ->notify(new ScraperBrokenNotification($source));
    }
} finally {
    $source->update(['last_run_at' => now()]);
}
}
}

```

11.3 AI-assisted extraction

Government notification PDFs are unstructured. Rules handle the common shapes; AI handles the rest.

```

class NotificationExtractor
{
    public function extract(string $pdfText): array
    {
        $response = Http::withHeaders(['x-api-key' => config('services.anthropic.key')])
            ->post('https://api.anthropic.com/v1/messages', [
                'model' => 'claude-sonnet-4-6',
                'max_tokens' => 2000,
                'messages' => [
                    'role' => 'user',
                    'content' => <<<PROMPT
                        Extract structured data from this Indian government job notification.
                        Return ONLY valid JSON, no markdown, no preamble, with these keys:
                        title, organisation, total_vacancies, min_qualification,
                        min_age, max_age, age_relaxation (object by category),
                        apply_start_date, apply_end_date (YYYY-MM-DD),
                        application_fee (object by category), salary_min, salary_max.
                        Use null for anything not clearly stated. Never guess a date.

                        NOTIFICATION TEXT:
                        {$pdfText}
                        PROMPT,
                    ],
                ],
            ]);

        $text = collect($response->json('content'))
            ->firstWhere('type', 'text')['text'] ?? '{}';

        return json_decode(trim(str_replace(['```json', '```'], '', $text)), true) ?? [];
    }
}

```

"Never guess a date" is load-bearing. A hallucinated deadline is the worst failure this product can have. Everything the extractor produces goes to human review regardless.

11.4 Source list (Phase 1)

Source	Frequency	Priority
tgpsc.gov.in	15 min	Critical
psc.ap.gov.in	15 min	Critical
ssc.gov.in	30 min	High
indianrailways.gov.in / RRB regionals	30 min	High
ibps.in	60 min	Medium
ncs.gov.in	60 min	Medium
TS/AP police recruitment boards	30 min	Critical (seasonal)
State PSU sites (SCCL, TSSPDCL, APSPDCL)	120 min	Medium
District collectorate sites	Daily	Low

Always keep a manual entry path. If a scraper breaks on notification day, a human must be able to publish in five minutes through Filament.

Chapter 12 — Search

సారాంశం: ప్రతి భాషకు వేరే ఇండెక్స్. కలిపితే వెతుకులాట పాడవుతుంది.

```
// app/Models/ExamNotification.php
public function searchableAs(): string
{
    return 'notifications_' . app()->getLocale(); // separate index per locale
}

public function toSearchableArray(): array
{
    $locale = app()->getLocale();

    return [
        'id'           => $this->id,
        'title'        => $this->getTranslation('title', $locale),
        'description'   => strip_tags($this->getTranslation('description', $locale) ?? ''),
        'organisation' => $this->organisation,
        'exam_name'     => $this->exam?->getTranslation('name', $locale),
        'job_type'      => $this->job_type,
        'qualification' => $this->min_qualification,
        'end_date'      => $this->apply_end_date?->timestamp,
        'published_at' => $this->published_at?->timestamp,
    ];
}
```

Index settings per locale:

```
$client->index('notifications_te')->updateSettings([
    'searchableAttributes' => ['title', 'exam_name', 'organisation', 'description'],
    'filterableAttributes' => ['job_type', 'qualification', 'end_date'],
    'sortableAttributes'   => ['published_at', 'end_date'],
    'rankingRules'         => ['words', 'typo', 'proximity', 'attribute',
                              'sort', 'exactness', 'published_at:desc'],
]);
```

Why separate indexes: Telugu tokenisation differs fundamentally from English. A shared index makes relevance scoring incoherent for both languages. The storage cost of duplication is trivial; the relevance cost of sharing is not.

Chapter 13 — Notifications and Push

సారాంశం: వాట్సాప్ మన బలమైన మార్గం. అతిగా పంపితే అందరూ ఆఫ్ చేసేస్తారు.

13.1 Channels

Channel	Use for	Notes
Web push (FCM)	Daily quiz, breaking notifications	Free, works in PWA
WhatsApp Cloud API	Daily quiz link, deadline reminders	Highest open rate; template approval needed
In-app	Everything	Always the fallback
Email	Weekly digest only	Low engagement in this segment

13.2 Frequency discipline

Type	Max frequency	Timing
Daily quiz	1/day	07:00 IST
New notification (matching user's exams)	Up to 3/day	Immediate
Deadline reminder	1 per saved item	3 days, then 1 day before
Streak warning	1/day	21:00 IST, only if not yet active today
Community reply	Batched	Max 1 batch per 4 hours

Hard cap: 5 pushes per user per day. Over-notification is the fastest way to get permanently muted, and once muted a user is effectively lost.

13.3 Batched dispatch

```
class SendNotificationAlert implements ShouldQueue
{
    public function handle(): void
    {
        User::whereHas('examPreferences', fn ($q) => $q->where('exam_id', $this->notification->exam_id))
            ->whereHas('pushTokens', fn ($q) => $q->where('is_active', true))
            ->chunkById(500, function ($users) {
                foreach ($users as $user) {
                    if (! app(PushBudget::class)->allows($user)) continue; // daily cap

                    $locale = $user->preferred_locale;
```

```
dispatch(new SendPush(  
    user: $user,  
    title: $this->notification->getTranslation('title', $locale),  
    body: __('New notification for :exam',  
        ['exam' => $this->notification->exam?->getTranslation('name',  
$locale)],  
        $locale),  
    url: route('notifications.show', [  
        'locale' => $locale, 'slug' => $this->notification->slug,  
    ]),  
))->onQueue('push');  
    }  
});  
}  
}
```

Chapter 14 — Performance and Scaling

సారాంశం: నోటిఫికేషన్ రోజున ట్రాఫిక్ 50 రెట్లు పెరుగుతుంది. దానికి ముందే సిద్ధం కావాలి.

14.1 Caching strategy

Content	Where	TTL	Invalidation
Exam hub pages	Cloudflare edge	6 hours	Purge on model save
Notification detail	Cloudflare edge	1 hour	Purge on save
Feed (anonymous)	Cloudflare edge	5 min	Time-based
Feed (logged in)	Redis, per user	2 min	On new notification
Eligibility result	Redis	1 hour	On profile change
Leaderboard	Redis sorted set	Live	—
Search results	Meilisearch internal	—	—

```
// Model observer – never let a stale exam page persist after an edit
class ExamObserver
{
    public function saved(Exam $exam): void
    {
        foreach (array_keys(config('locales.supported')) as $locale) {
            Cache::forget("exam:{$exam->slug}:{$locale}");
            dispatch(new PurgeCloudflareUrl(
                route('exams.show', ['locale' => $locale, 'slug' => $exam->slug])
            ));
        }
    }
}
```

14.2 Scaling ladder

Users	Setup	Est. monthly cost
0–50k MAU	1 VPS (4 vCPU, 8 GB): app + MySQL + Redis + Meilisearch	₹2,000
50k–300k	2 app servers behind a load balancer; DB on its own box	₹8,000
300k–1M	3 app servers; MySQL primary + read replica; separate Redis and Meilisearch	₹45,000
1M–5M	Autoscaling app tier; managed MySQL; Redis cluster; read replicas per region	₹2,50,000

14.3 Notification-day playbook

This is the operational document to have printed and pinned.

T-24 hours (notification expected) - Scale app servers to 3x - Raise Cloudflare cache TTLs - Pre-create the exam hub page and draft notification record - Prepare the push message and get it approved in advance - Put the content team on standby

T-0 (notification released) - Publish within 15 minutes - Push in staggered waves of 50,000 users, not all at once - Watch: response time p95, queue depth, error rate, DB connections

During the spike - If p95 response exceeds 2s → enable static fallback mode (a cached HTML page with the essential facts, no personalisation) - If DB connections saturate → shed the personalised feed, serve the anonymous cached feed to everyone - Never let the site go fully down; degraded is always better than absent

T+48 hours - Scale back down - Post-mortem: what broke, what was slow, what to fix before next time

14.4 Frontend performance

- Critical CSS inlined, rest deferred
 - Images: WebP, lazy-loaded, explicit width/height to avoid layout shift
 - Telugu font subset, preloaded
 - Livewire lazy-loading for below-the-fold components
 - Service worker caches the app shell, saved material, and last-seen feed
 - **Budget: under 100 KB JS on first load.** Enforce it in CI with a bundle-size check that fails the build.
-

Chapter 15 — Security

సారాంశం: లక్షల మంది వ్యక్తిగత సమాచారం మన దగ్గర ఉంటుంది. దానికి బాధ్యత మనదే.

15.1 Application security

- Laravel's CSRF, XSS escaping, and query bindings — never build raw SQL from user input
- Rate limiting: 60 req/min general, 5/min on OTP, 10/hour on uploads
- File uploads: validate MIME by content not extension; store outside the web root; serve through signed URLs
- Admin panel behind a separate subdomain, IP allowlist, and mandatory 2FA
- All secrets in environment variables; never in the repo
- Dependency scanning via Dependabot; monthly `composer audit`

15.2 DPDP Act compliance — build in v1

Requirement	Implementation
Consent	Itemised checkboxes at signup, in the user's language, with purpose stated. Consent version and timestamp stored.
Purpose limitation	Profile data is used only for eligibility matching and personalisation. Say so, then honour it.
Right to access	Working "Download my data" — generates a JSON export
Right to erasure	Working "Delete my account" — hard-deletes personal data within 30 days, retains only anonymised aggregates
Data minimisation	No Aadhaar, no caste certificates, no full address. District is enough.
Minors	Minimum age 18 at signup, stated in terms. Our audience is graduates; there is no reason to collect data on children.
Breach notification	Documented process, named responsible person, before launch
Retention	Inactive accounts anonymised after 3 years

```
class DataExportService
{
    public function export(User $user): array
    {
        return [
```

```
'profile'      => $user->profile?->toArray(),
'preferences'  => $user->examPreferences->pluck('slug'),
'quiz_attempts' => $user->quizAttempts()->select('score','total_marks','created_at')->get(),
'posts'        => $user->posts()->select('title','body','created_at')->get(),
'saved'        => $user->savedNotifications()->pluck('slug'),
'exported_at'  => now()->toIso8601String(),
    ];
}
```

15.3 Content security

- Every uploaded file scanned before publication
 - Community posts run through a profanity and spam filter, then human review on flag
 - Phone numbers auto-redacted from public posts — this space is full of "guaranteed job" scams
 - Full audit log on every admin action touching user data
-

Chapter 16 — Testing

సారాంశం: అర్హత లాజిక్కు, తేదీలకు టెస్టులు తప్పనిసరి. మిగతా వాటికి కనీసం స్మోక్ టెస్టులు.

16.1 Priorities

Area	Coverage target	Why
Eligibility engine	95%	A wrong answer here misleads a user about their career
Streak logic	90%	Edge cases around midnight, timezones, freezes
Translation fallback	90%	Silent failures show English to Telugu users
Scraper parsers	80%	Government sites change without warning
Payment/subscription	95%	Money
UI components	Smoke tests	Diminishing returns

16.2 Sample tests

```
it('applies category age relaxation correctly', function () {
  $notification = ExamNotification::factory()->create([
    'max_age' => 30,
    'age_relaxation' => ['obc' => 3, 'sc' => 5, 'st' => 5],
    'age_reference_date' => '2026-07-01',
  ]);

  $profile = Profile::factory()->create([
    'date_of_birth' => '1993-06-01', // 33 on reference date
    'category' => 'sc', // 30 + 5 = 35 → eligible
  ]);

  expect(app(EligibilityService::class)->check($notification, $profile)['status'])
    ->toBe('eligible');
});

it('never loses a streak within the freeze window', function () {
  $user = User::factory()->create();
  Streak::factory()->for($user)->create([
    'current_streak' => 10,
    'last_active_date' => today()->subDays(2),
    'freezes_left' => 1,
  ]);

  expect(app(StreakService::class)->record($user)['streak'])->toBe(11);
  expect($user->streak->fresh()->freezes_left)->toBe(0);
});

it('falls back to english when a telugu translation is missing', function () {
  app()->setLocale('te');
```

```
$exam = Exam::factory()->create(['name' => ['en' => 'SSC CGL']]);  
  
expect($exam->name)->toBe('SSC CGL');  
});
```

16.3 Load testing

Before every notification season, run k6 against staging at 50x expected baseline. Test specifically: - The feed endpoint (highest traffic) - The notification detail page - Push dispatch throughput - The DB connection pool under concurrent eligibility checks

Chapter 17 — CI/CD and Deployment

సారాంశం: ఆటోమేటిక్ డిప్లాయ్. మాన్యువల్ గా చేస్తే ఏదో ఒకరోజు తప్పు జరుగుతుంది.

17.1 CI

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8
        env: { MYSQL_ROOT_PASSWORD: root, MYSQL_DATABASE: testing }
        options: >-
          --health-cmd="mysqladmin ping" --health-interval=10s --health-retries=5
      redis:
        image: redis:7
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3', extensions: mbstring, intl, pdo_mysql, redis }
      - run: composer install --prefer-dist --no-progress
      - run: cp .env.testing .env && php artisan key:generate
      - run: php artisan migrate --force
      - run: ./vendor/bin/pint --test
      - run: ./vendor/bin/phpstan analyse --memory-limit=1G
      - run: php artisan test --parallel
      - run: npm ci && npm run build
      - name: Enforce JS bundle budget
        run: node scripts/check-bundle-size.js # fails above 100 KB gzipped
```

17.2 Deploy

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    needs: []
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.VPS_HOST }
          username: ${ secrets.VPS_USER }
          key: ${ secrets.VPS_SSH_KEY }
```

```
script: |
  cd /var/www/sadhana
  php artisan down --render="errors::503" --retry=30
  git pull origin main
  composer install --no-dev --optimize-autoloader
  php artisan migrate --force
  npm ci && npm run build
  php artisan config:cache
  php artisan route:cache
  php artisan view:cache
  php artisan queue:restart
  sudo supervisorctl restart sadhana-worker:*
  php artisan up
```

17.3 Supervisor for queue workers

```
[program:sadhana-worker]
command=php /var/www/sadhana/artisan queue:work redis --queue=high,push,default,low --sleep=3 --tries=3
--max-time=3600
autostart=true
autorestart=true
numprocs=4
user=www-data
redirect_stderr=true
stdout_logfile=/var/www/sadhana/storage/logs/worker.log
stopwaitsecs=3600
```

Queue priority matters: **high** (scraper alerts), **push** (time-sensitive), **default**, **low** (translation, image generation).

17.4 Scheduled tasks

```
// routes/console.php
Schedule::command('scrape:run')->everyThirtyMinutes();
Schedule::command('quiz:publish')->dailyAt('06:45')->timezone('Asia/Kolkata');
Schedule::job(new SendDailyQuizPush)->dailyAt('07:00')->timezone('Asia/Kolkata');
Schedule::job(new SendStreakReminders)->dailyAt('21:00')->timezone('Asia/Kolkata');
Schedule::command('notifications:expire')->dailyAt('00:30');
Schedule::command('sitemap:generate')->dailyAt('02:00');
Schedule::command('analytics:rollup')->hourly();
Schedule::command('ad-events:partition')->monthlyOn(1, '03:00');
Schedule::command('backup:run')->dailyAt('03:30');
```

17.5 Backups

- MySQL: daily full dump to R2, retained 30 days; binlogs for point-in-time recovery
- R2 files: versioning enabled
- **Restore drill quarterly.** An untested backup is not a backup.

Chapter 18 — Monitoring

సారాంశం: ఏదైనా పాడైతే మనం మొదట తెలుసుకోవాలి, విద్యార్థి కాదు.

Tool	Watches
Sentry	Exceptions, with release tracking
Laravel Pulse	Slow queries, slow jobs, queue depth, cache hit rate
Uptime Kuma	Homepage, feed, a representative exam page, admin — 1-min interval
Google Search Console	Indexing per language, ranking, Core Web Vitals
Plausible or GA4	Traffic, funnels, retention
Custom dashboard	Notification latency, translation backlog, moderation queue depth

Alert thresholds (to phone, not email): - Error rate above 1% for 5 minutes - Response p95 above 2s for 5 minutes - Queue depth above 10,000 - Any scraper failing 3 consecutive runs - Moderation queue above 100 items - Notification published more than 4 hours after the official release

Chapter 19 — Launch Checklist

సారాంశం: ఈ జాబితా పూర్తయ్యాకే లైవ్ కి వెళ్లాలి.

Content

- ☐ 30 exam hub pages complete in Telugu and English
- ☐ 12 months of historical notifications loaded
- ☐ 500 questions in the quiz pool, both languages
- ☐ 60 days of daily quizzes pre-scheduled
- ☐ 50 official material PDFs uploaded and categorised

Technical

- ☐ Load tested at 50x baseline
- ☐ Sitemaps generated and submitted per language
- ☐ hreflang verified with Google's testing tool
- ☐ Structured data validated (JobPosting, Course, FAQPage)
- ☐ PWA installable, service worker caching correctly
- ☐ LCP under 2.5s on a real budget Android on 3G
- ☐ Backups running and one restore tested
- ☐ Sentry, Pulse, uptime monitoring live with alerts routed to a phone

Legal

- ☐ Privacy policy and terms published, in both languages
- ☐ DPDP consent flow live
- ☐ Data export and account deletion working end to end
- ☐ Copyright policy and takedown process published with a real contact
- ☐ Disclaimers on every notification page

Growth

- ☐ WhatsApp channels created and seeded with subscribers
- ☐ YouTube and Instagram accounts live with 5 posts each
- ☐ 20 beta users onboarded in person and giving feedback
- ☐ Google Search Console and Analytics verified

- ☐ Launch-day notification drafted and approved in advance
-

Chapter 20 — Sprint Plan

సారాంశం: పద్నాలుగు వారాలు. ప్రతి వారం ఏమి చేయాలో ఇక్కడ ఉంది.

Sprint 0 — Foundation (Weeks 1–2)

Day	Task
1–2	Competitor teardown on a real budget phone (Volume 1, Chapter 4). Screenshot everything.
3	Repo, Laravel 12 install, Docker/Sail, GitHub Actions skeleton
4	Database migrations — all core tables from Chapter 4
5	Models, relationships, factories, seeders
6–7	i18n: config, routing, <code>SetLocale</code> , language files, Telugu typography
8	Filament install, first admin resources (Exam, Notification)
9	Design system: colours, type scale, base components, tested with Telugu text
10	Auth: phone OTP, profile creation

Deliverable: an admin can create a bilingual exam and see it render correctly in Telugu on a phone.

Sprint 1 — Exam Hub (Weeks 3–4)

Task
Exam listing and detail pages, all 11 sections
SEO: meta, canonical, hreflang, sitemaps
Structured data: Course, FAQPage
Filament resources for syllabus, pattern, cutoffs
Content sprint: populate 10 exams fully
Cloudflare caching and purge-on-save observer

Deliverable: 10 exam pages live and indexable in both languages.

Sprint 2 — Notifications (Weeks 5–7)

Task
Notification model, admin CRUD, publish workflow
Eligibility engine with full test coverage
Feed component with filters and pagination
Notification detail page with JobPosting schema
Save and remind
Meilisearch integration, per-locale indexes
First two scrapers (TGPSC, APPSC) with the review queue
Content sprint: 12 months of historical notifications

Deliverable: a user sets a profile and sees a personalised, eligibility-tagged feed.

Sprint 3 — Quiz and Streaks (Weeks 8–10)

Task
Question bank, bilingual admin entry
Daily quiz generation command with pool-depletion alerting
Quiz UI with language toggle per question
Streak service including freeze logic
Redis leaderboards, state and district
Shareable result image
FCM web push
WhatsApp Cloud API integration and template approval
Content sprint: 500 questions, 60 days scheduled

Deliverable: the 7 AM habit loop works end to end.

Sprint 4 — Polish and Launch (Weeks 11–14)

Task
PWA: manifest, service worker, install prompt, offline fallback
Performance pass against the budgets in Chapter 1.3
Translation review panel in Filament
Privacy policy, terms, consent flow, data export, account deletion
Analytics and event tracking
Load test at 50x
Sentry, Pulse, uptime alerts
Beta with 20 real users, in person
Fix everything they trip over
Launch

Deliverable: live, on a day a major notification is expected.

Post-launch cadence

- **Daily:** check DAU, quiz completions, notification latency, error rate
- **Weekly:** cohort retention review; ship one improvement per day based on feedback
- **Week 14 gate:** if D7 retention is below 20%, **stop all feature work and fix the loop.** Do not start Phase 2.

Appendix A — Environment variables

```
APP_NAME=Sadhana
APP_ENV=production
APP_URL=https://sadhana.study
APP_LOCALE=te
APP_FALLBACK_LOCALE=en

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_DATABASE=sadhana
DB_USERNAME=
DB_PASSWORD=

REDIS_HOST=127.0.0.1
CACHE_STORE=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis

SCOUT_DRIVER=meilisearch
MEILISEARCH_HOST=http://127.0.0.1:7700
MEILISEARCH_KEY=

FILESYSTEM_DISK=r2
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET=sadhana
R2_ENDPOINT=
R2_URL=https://cdn.sadhana.study

FCM_SERVER_KEY=
WHATSAPP_PHONE_NUMBER_ID=
WHATSAPP_ACCESS_TOKEN=
WHATSAPP_VERIFY_TOKEN=

ANTHROPIC_API_KEY=

CLOUDFLARE_ZONE_ID=
CLOUDFLARE_API_TOKEN=

SENTRY_LARAVEL_DSN=
OPS_EMAIL=
OPS_PHONE=
```

Appendix B — Composer dependencies

```
{
  "require": {
    "php": "^8.3",
    "laravel/framework": "^12.0",
    "livewire/livewire": "^3.5",
    "filament/filament": "^3.2",
    "spatie/laravel-translatable": "^6.7",
    "laravel/scout": "^10.0",
    "meilisearch/meilisearch-php": "^1.8",
    "spatie/laravel-sitemap": "^7.2",
    "spatie/laravel-backup": "^9.0",
    "intervention/image": "^3.6",
    "smalot/pdfparser": "^2.10",
    "league/flysystem-aws-s3-v3": "^3.0",
    "sentry/sentry-laravel": "^4.6",
    "propaganistas/laravel-phone": "^5.3"
  },
  "require-dev": {
    "pestphp/pest": "^3.0",
    "laravel/pint": "^1.17",
    "phpstan/phpstan": "^1.11",
    "barryvdh/laravel-debugbar": "^3.13"
  }
}
```

Appendix C — Definition of Done

A task is done when:

- ☐ Code merged to `main` and deployed to staging
 - ☐ Tests written and passing; eligibility and streak logic at target coverage
 - ☐ Works correctly in Telugu **and** English — verified, not assumed
 - ☐ Tested on a real budget Android on a throttled connection
 - ☐ Meets the performance budgets in Chapter 1.3
 - ☐ Cache keys include locale
 - ☐ SEO tags present where the page is public
 - ☐ Accessible: keyboard navigable, adequate contrast, labelled controls
 - ☐ Admin can manage it through Filament where relevant
 - ☐ Errors reported to Sentry, not swallowed
-

End of Volume 2.

Volume 1 explains why to build this. Volume 2 explains how. The only thing left is Sprint 0, Day 1.